Efficiency-Directed Agentic Vibe-Coding: A Research Summary

Ambiguity Resolution, Agent Feedback, End-Result Productivity, and Independent Quality Assurance

Samuel A. Gray III

October 11, 2026 · 15:00 MDT

Authorship statement. Samuel A. Gray III is the sole author. Research organization, drafting, mathematical typesetting, and editorial assistance were provided by ChatGPT (OpenAI). This document summarizes the author's October 8–11, 2026 research propositions and ensuing human–AI dialogue. **Status:** conceptual research proposal; the proposed experiments have not been performed.

Abstract

The paper proposes that software-generating agents should seek *efficient end-result productivity*, not simply low coding cost or high code volume. It develops a progression from direct vibe-coding to ambiguity resolution, iterative feedback, and generalized efficiency-directed feedback for open-weight agentic models. It distinguishes (i) development costs, (ii) the useful output of deployed software, (iii) independent quality-assurance (QA) evidence, and (iv) lifecycle value efficiency. It recommends a staged research program comparing feedback policies and model modifications under matched tasks and rigorous independent evaluation. Existing studies show that the effects of AI-assisted development depend on context and cannot establish universal superiority for any methodology.[1][2]

1. Research question and competing approaches

The initial claim challenges the procedural overhead of conventional software development: if a human can express a problem precisely—ideally through mathematics—why should an AI agent recreate extensive process before solving it? A counterargument is that verification and formal process may reduce costly errors. Both positions lead to a falsifiable question: **which combination of specification rigor, feedback, implementation autonomy, and QA produces the greatest verified product value for a given resource budget?**

We distinguish a *nonimplementation/critique* baseline (**C**, useful primarily for decision analysis), conventional structured development (**T**), unclarified direct vibe-coding (**V0**), initial ambiguity resolution (**V1**), iterative clarification feedback (**V2**), and generalized productivity-aware feedback (**V3**). C is not a fair coding-speed comparator because it does not aim to deliver the same software artifact.

2. Separate cost, delivered productivity, and QA

Let C_B be build cost *excluding QA*, C_{QA} independent testing/verification cost, $C_O(T)$ operating cost over horizon T, and $C_M(T)$ maintenance and repair cost. Then development cost and total lifecycle cost are

$$C_D = C_B + C_{QA}, \qquad C_L(T) = C_B + C_{QA} + C_O(T) + C_M(T).$$

Let $Y(T)$ be *useful, correct operations or task outcomes actually delivered* during the observation period and $V(T)$ their measured user/economic value. End-result productivity can be measured as throughput $P_E(T) = \mathbb{E}[Y(T)]/T$, and

[1] Becker, J., Rush, N., Barnes, B., and Rein, D. (2025). *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.* METR. Randomized study of 16 experienced developers and 246 tasks; AI access associated with 19% longer completion time in this setting. https://metr.org/Early_2025_AI_Experienced_OS_Devs_Study-paper.pdf

[2] Google Cloud / DORA (2025). *Announcing the 2025 DORA Report.* Reports positive relationships between AI adoption and delivery throughput/product performance, while indicating a negative relationship with delivery stability. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report

(when runtime input consumption can be measured consistently) resource productivity as $P_R(T) = \mathbb{E}[Y(T)]/\mathbb{E}[R(T)]$. Lifecycle value efficiency is

$$\boxed{\eta_L(T) = \frac{\mathbb{E}[V(T)]}{\mathbb{E}[C_L(T)]}}.$$

The ratio of expectations is not the same as the expectation of an individual project's value-to-cost ratio. Comparisons must use compatible value units, horizons, task mixes, and uncertainty intervals. Optimizing cost alone can yield a cheap but unproductive product; optimizing throughput alone can yield a fast but incorrect or unreliable product.

QA is an independent evidence-producing activity, not an arbitrary quality multiplier. Hidden tests, security checks, reliability trials, performance benchmarks, and user acceptance tests estimate whether reported output is truly useful. Defect correction changes the product; QA evidence changes our confidence in its quality. In the above accounting, QA is included exactly once.

3. Ambiguity and interactive feedback

For an input prompt P, let S denote the user's intended observable behavior and $p(s \mid P)$ a modeled distribution over plausible intended specifications. Conditional entropy represents *estimated* uncertainty:

$$H(S \mid P) = -\sum_s p(s \mid P) \log_2 p(s \mid P).$$

The agent can ask a question q and obtain an authorized response a. It should not equate a confidently selected assumption with a clarified requirement. Meaningful ambiguity reduction occurs only when new information better identifies user intent. The objective is **not** $H \to 0$ at any cost: harmless ambiguities can reasonably remain.

For a proposed question q, define its *expected net feedback value* as

$$\mathrm{EVF}(q) = \mathbb{E}[C_{\mathrm{down}} \mid \text{no question}] - \mathbb{E}[C_{\mathrm{down}} + C_q \mid \text{question } q],$$

where downstream costs include expected defects, rework, deployment losses, and relevant quality consequences; C_q includes human time, tokens, and delays. Under cost minimization, ask when $\mathrm{EVF}(q) > 0$, subject to mandatory safety or correctness rules. For value-maximizing cases, questions should instead be ranked by expected *net change in product value minus full lifecycle cost*. Estimates must be calibrated against observed outcomes rather than treated as known probabilities.

4. V3: Generalized efficiency-directed feedback

An advanced agent should proactively recognize more than prompt ambiguity: unsupported assumptions; missing acceptance criteria; incorrect decomposition; wasteful context/tokens; redundant tool calls; model–task mismatch; poor algorithmic choices; inadequate or excessive tests; fragile interfaces; operational bottlenecks; security risks; and premature stopping. The agent chooses among actions such as **ask, specify, implement, test, revise, delegate, or stop** according to expected downstream results.

Formally, let a feedback policy π map the observed project state to a next action. Study

$$\max_{\pi} \ \eta_L(\pi; T) \quad \text{subject to} \quad \Pr(\text{critical failure} \mid \pi) \le \epsilon, \quad Q_{QA}(\pi) \ge q_{\min}.$$

Here Q_{QA} denotes a *defined, independently assessed acceptance measure* rather than a self-reported model score. Because an efficiency ratio may hide tradeoffs, report the Pareto frontier over lifecycle cost, useful throughput, defect rates, reliability, and user value as well.

5. Recommendations for open-weight agentic AI

Tier A—no weight changes (first test): build a transparent feedback controller in the agent harness. Require structured records for uncertainty, proposed questions, expected impact, user-confirmed requirements, tool evidence, and stop decisions. Add token/time budgets; independent tests; and inexpensive model routing. This establishes whether feedback design helps before attributing gains to model weights.

Tier B—parameter-efficient weight adaptation: develop supervised fine-tuning/LoRA examples and preference data for consequential ambiguity detection, question selection, calibrated uncertainty, tool recovery, evidence citation, and refusing unsupported inferences. Evaluate on tasks and repositories not seen during training; monitor loss of general capability.

Tier C—verified-outcome optimization: where feasible, train policies against external test outcomes and lifecycle-aware reward functions in isolated sandboxes. Avoid reward hacking: the agent must not modify hidden tests, fabricate evidence, or optimize merely for test pass counts.

Tier D—exploratory architecture: investigate auxiliary uncertainty/cost prediction heads or learned feedback policies, but treat their benefits as unproven until ablations show improvement over Tier A.

"Open source" and "open weight" are not identical legal/technical categories; model license and reproducibility requirements must be assessed separately.

6. Falsifiable empirical protocol

Use matched, randomized software tasks spanning mathematical utilities, web services, bug fixes, integrations, and performance-sensitive programs. Cross **workflow** (T/V0/V1/V2/V3) with **specification type** (ordinary prose/formal contract), so the influence of mathematical precision is not mistaken for the influence of process. Replicate tasks across models, seeds, and difficulty levels. Hold compute/tool entitlements and evaluation standards constant, or explicitly model differences. Apply an independent QA harness—including withheld tests and where appropriate security and load evaluation—before calculating final performance.

Pre-register primary endpoints: (1) verified useful output and value over a fixed horizon, (2) build cost and separate QA cost, (3) full lifecycle cost, and (4) severe defect rate. Secondary endpoints: time to acceptance, tokens, clarification burden, rework, calibration of risk estimates, runtime resource use, and stability. Compare Tier A harness-only V3 with Tier B/C weight-adapted versions, then ablate ambiguity feedback, tool feedback, QA feedback, and stopping rules. Report confidence intervals and failed tasks, not only average successful completions. SWE-bench offers a reproducible starting benchmark, but by itself does not measure deployed value or full lifecycle productivity.[3]

7. Interpretation, limits, and research contribution

A 2025 randomized trial of experienced open-source developers found that access to then-current AI tools increased completion time by 19% within its particular study setting.[4] Google's 2025 DORA report describes positive associations with throughput and product performance alongside a negative association with delivery stability; these are not universally causal estimates.[5] These findings motivate task-specific experimentation and independent QA, rather than a presumption that either traditional engineering or vibe-coding invariably wins. NIST's generative-AI profile provides complementary guidance for risk-aware evaluation and controls.[6]

Proposed contribution: replace the binary "vibe-coding versus software engineering" debate with an empirically testable **feedback policy optimization** problem. The agent should formalize what matters, question consequential uncertainties, preserve verified evidence, and optimize useful deployed outcomes under explicit risk and cost constraints. This is a research agenda, **not an established demonstration** that V3 is superior.

---

[3] Jimenez, C. E., et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. https://openreview.net/forum?id=VTF8yNQM66 ; project and reproducibility materials: https://github.com/SWE-bench/SWE-bench

[4] Becker, J., Rush, N., Barnes, B., and Rein, D. (2025). *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.* METR. Randomized study of 16 experienced developers and 246 tasks; AI access associated with 19% longer completion time in this setting. https://metr.org/Early_2025_AI_Experienced_OS_Devs_Study-paper.pdf

[5] Google Cloud / DORA (2025). *Announcing the 2025 DORA Report.* Reports positive relationships between AI adoption and delivery throughput/product performance, while indicating a negative relationship with delivery stability. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report

[6] Autio, C., et al. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile.* NIST AI 600-1. https://doi.org/10.6028/NIST.AI.600-1